

50 NODE.JS

INTERVIEW QUESTIONS AND ANSWERS





Q 1. What is Node.js?

ANS: Node.js is a JavaScript runtime built on the V8 engine. It allows JavaScript to run on the server side. It is fast and event-driven.

Q 2. What is NPM?

ANS: NPM is the default package manager for Node.js. It helps install libraries and manage dependencies. It also hosts millions of open-source packages.

Q 3. What is Event Loop in Node.js?

ANS: The event loop handles asynchronous tasks in Node.js. It allows non-blocking operations. This makes Node.js highly scalable.

Q 4. What is non-blocking I/O?

ANS: Non-blocking I/O means tasks run without waiting for others to complete. Node.js uses this for fast performance. It helps handle many requests at once.



Q 5. What are Node.js modules?

ANS: Modules are reusable blocks of code in Node.js. They help organize and structure applications. They can be built-in, custom, or third-party.

Q 6. What is CommonJS?

ANS: CommonJS is the module system used by Node.js (require/export). It loads modules synchronously. It's used in server-based applications.

Q 7. Difference between require() and import?

ANS: require() is CommonJS and works synchronously. import is ES6 and works asynchronously. Modern Node.js supports both.

Q 8. What is Express.js?

ANS: Express.js is a web framework for Node.js. It simplifies routing and middleware handling. It helps build APIs quickly.



Q 9. What is middleware in Express?

ANS: Middleware is a function that handles requests before the response. It can modify request/response objects. It's used for auth, logging, etc.

Q 10. What is package.json?

ANS: package.json stores project metadata, dependencies, and scripts. It is required for every Node.js project. It helps manage and run the application.

Q 11. What is callback in Node.js?

ANS: A callback is a function executed after another function completes. Node.js heavily uses callbacks for async operations. It can sometimes cause callback hell.

Q 12. What is callback hell?

ANS: Callback hell occurs when multiple callbacks are nested. It makes code difficult to read and manage. Promises and async/await solve this.



Q 13. What is a Promise?

ANS: A promise represents an async operation result. It has states like pending, fulfilled, and rejected. It helps avoid callback hell.

Q 14. What is async/await?

ANS: Async/await makes asynchronous code look synchronous. It simplifies promise handling. It improves readability.

Q 15. What is Streams in Node.js?

ANS: Streams process data in chunks instead of loading it fully. It improves performance for large data. Streams are readable, writable, or both.

Q 16. What is Buffer in Node.js?

ANS: Buffer is used to store binary data in Node.js. It helps handle raw data from streams or files. It is faster than using strings for binary data.



Q 17. What is the difference between `process.nextTick()` and `setImmediate()`?

ANS: `process.nextTick()` executes before the next event loop iteration. `setImmediate()` executes on the next iteration. Both are used for async operations.

What is the difference between

Q 18. Node.js and JavaScript?

ANS: JavaScript runs in the browser. Node.js runs on the server. Node.js adds features like file system and network access.

Q 19. What are the types of API in Node.js?

ANS: Node.js APIs are of three types: File System API, HTTP API, and Stream API. They allow interaction with OS, network, and files.

Q 20. What is the difference between `spawn()` and `fork()`?

ANS: `spawn()` launches a new process with a command. `fork()` spawns a new Node.js process. `fork()` allows communication via IPC.



Q 21. What is clustering in Node.js?

ANS: Clustering allows Node.js to create multiple worker processes. Each worker runs on a separate CPU core. It improves performance on multi-core machines.

Q 22. What is REPL in Node.js?

ANS: REPL stands for Read-Eval-Print Loop. It allows executing Node.js commands interactively. Useful for testing code snippets quickly.

Q 23. What is the difference between `readFile()` and `createReadStream()`?

ANS: `readFile()` reads the entire file into memory.
`createReadStream()` reads data in chunks.
Streams are better for large files.

Q 24. What is the difference between `process.env` and config files?

ANS: `process.env` stores environment variables.
Config files store application settings. Env variables are preferred for sensitive info.



Q 25. What is the difference between PUT and PATCH?

ANS: PUT updates the entire resource. PATCH updates only specific fields. Both are used in REST APIs.

Q 26. How do you handle errors in Node.js?

ANS: Errors can be handled using callbacks, try/catch, or promises. Express apps often use error-handling middleware. Proper handling prevents crashes.

Q 27. What is CORS?

ANS: CORS (Cross-Origin Resource Sharing) allows resources to be shared between different origins. Browsers enforce CORS for security. Express has middleware to handle it.

Q 28. What is JWT in Node.js?

ANS: JWT (JSON Web Token) is used for authentication. It securely transmits info between client and server. Often used with APIs.



Q 29. What is the difference between synchronous and asynchronous code?

ANS: Synchronous code runs line by line, blocking the next operation. Asynchronous code runs without waiting. Node.js uses async code for efficiency.

Q 30. How do you debug Node.js applications?

ANS: Node.js apps can be debugged using `console.log()`, the built-in debugger, or Chrome DevTools. VS Code also has debugging tools. It helps identify issues quickly.

Q 31. What is the difference between HTTP and HTTPS?

ANS: HTTP is unsecured while HTTPS is secured using SSL/TLS. HTTPS encrypts data between client and server. Most APIs and websites use HTTPS for security.



Q 32. What is a REST API?

ANS: REST API allows communication between client and server using HTTP methods. It is stateless and uses JSON/XML. Widely used in web applications.

Q 33. What is the difference between `process.on('exit')` and `process.on('beforeExit')`?

ANS: `process.on('exit')` triggers when Node.js exits, no async allowed. `process.on('beforeExit')` triggers before exit, async tasks can run. Useful for cleanup.

Q 34. What is the difference between `res.send()` and `res.json()`?

ANS: `res.send()` sends any response type. `res.json()` specifically sends JSON and sets content-type automatically. `res.json()` is preferred for APIs.

Q 35. How do you handle file uploads in Node.js?

ANS: File uploads can be handled using middleware like Multer. It parses multipart/form-data requests. Files can then be saved on disk or cloud storage.



Q 36. What is the difference between

ANS: cluster and worker threads?

Cluster creates multiple Node.js processes to handle load. Worker threads allow multi-threading in a single process. Both improve performance differently.

Q 37. What is the difference between

ANS: setTimeout() and setInterval()?

setTimeout() runs a function once after a delay. setInterval() runs a function repeatedly at intervals. Both are asynchronous timers in Node.js.

Q 38. What is the difference between cluster and PM2?

ANS: Cluster is a Node.js module for multi-core process handling. PM2 is a process manager for production, supporting clustering and monitoring. PM2 simplifies deployment.

Q 39. How does Node.js handle child

ANS: processes?

Node.js can create child processes using spawn(), fork(), or exec(). Child processes allow parallel execution of tasks. Useful for CPU-intensive operations.



Q 40. What is the difference between synchronous and asynchronous file operations?

ANS: Synchronous file operations block execution until completed. Asynchronous operations don't block, using callbacks or promises. Async is preferred for scalability.

Q 41. What is a template engine in Node.js?

ANS: Template engines like EJS, Pug generate HTML dynamically. They help render data on server-side. Useful for building dynamic web pages.

Q 42. What is Node.js REPL?

ANS: REPL (Read-Eval-Print Loop) lets you run Node.js commands interactively. Useful for testing code snippets. Available via terminal by typing node.

Q 43. What is the difference between local and global modules in Node.js?

ANS: Local modules are installed per project using npm. Global modules are installed system-wide. Global modules can be used in any project.



Q 44. What is the difference between `cluster.fork()` and `child_process.fork()`?

ANS: `cluster.fork()` creates multiple workers for load balancing. `child_process.fork()` creates a single Node.js child process. Both allow inter-process communication.

Q 45. What are Node.js timers?

ANS: `setTimeout()`, `setInterval()`, and `setImmediate()` schedule functions to run later. They are part of the Node.js event loop. Useful for async scheduling.

Q 46. What is the difference between `cluster module` and `worker_threads module`?

ANS: Cluster creates multiple Node.js processes for parallelism. `worker_threads` allows multi-threading within a single process. `worker_threads` share memory while `clusters` do not.

Q 47. What is `process.argv` in Node.js?

ANS: `process.argv` stores command-line arguments passed to a Node.js program. First two elements are node path and script path. Useful for CLI tools.



Q 48. How does Node.js handle memory management?

ANS: Node.js uses V8 engine for memory management. It automatically handles allocation and garbage collection. Developers should avoid memory leaks.

Q 49. What is the difference between `require()` and `require.resolve()`?

ANS: `require()` loads a module and executes it. `require.resolve()` returns the module's resolved path without executing it. Useful for checking module paths.

Q 50. What is the difference between Node.js and PHP?

ANS: Node.js is asynchronous and event-driven. PHP is synchronous and blocking. Node.js is better for real-time applications, while PHP is widely used for web pages.

The End



Lokesh Mali

Website Developer

My **CONTENT**

HTML

CSS

BOOTSTRAP

JAVASCRIPT

MERN

NEXT.JS

WEB DEVELOPMENT

INTERVIEW